

CSE 6140 / CX 4140

Computational Science & Engineering Algorithms

Project - TSP

Group 6: Xingjian Tao, Yijun Lai, Yida Cheng, Zhaoyu Xu
Georgia Tech: CSE 6140
Instructor: Nabil Imam

December 1st, 2024

GitHub Repository

<https://github.com/hubery-cao/TSP-Solver/tree/main>

Usage

The usage adheres to the requirements outlined in the project description. It is important to note that you should provide the full path to the data file (with or without the .tsp extension). The output file will be automatically generated in the same directory as the specified data file.

1 Introduction

The Traveling Salesman Problem (TSP) involves finding the shortest possible route that visits each point exactly once and returns to the starting point. The TSP is NP-complete in its decision form and NP-hard in its optimization form, meaning that no polynomial-time algorithm exists to solve it exactly unless $P=NP$. While certain cases of TSP have additional structure that can be exploited, such as the Metric TSP, these variations remain NP-hard in their optimization form, and no exact polynomial-time algorithms have been discovered for them so far.

The Metric TSP is defined by edge costs that are symmetric and satisfy the triangle inequality, making it more structured than the general TSP. This project implements and compares three algorithms for solving the Metric TSP: an exact brute-force solution, a 2-approximation algorithm based on the Minimum Spanning Tree (MST), and a heuristic local search algorithm using Genetic Algorithms (GA). These algorithms represent distinct approaches to solving TSP, and the project evaluates their performance, efficiency, and solution quality.

2 Algorithms

2.1 Exact Algorithm (Brute-Force with Branch-and-Bound)

The Brute-Force with Branch-and-Bound is an exact algorithm for solving the Traveling Salesman Problem (TSP) by systematically exploring all possible routes while eliminating paths that cannot yield better solutions than the current best. Starting from an arbitrary root node:

- A priority queue of partial solutions is maintained, prioritized by a lowest bound on their total cost, which is estimated using an approximation method such as a Minimum Spanning Tree (MST).
- At each step, the algorithm selects the partial route with the lowest cost, extends it by adding unvisited cities while pruning unpromising ones, and updates the best known solution when a complete route with a lower cost is found. This process continues until all viable routes are examined or a predefined cutoff time is reached.

The worst-case time complexity of the Brute Force Algorithm for TSP is $O(n!)$, where n is the number of cities, due to the factorial growth of possible permutations. However, effective bounding and pruning strategies typically reduce the number of routes that need to be evaluated in practice, making the algorithm feasible for moderately sized instances.

The brute-force method was tested on different datasets with varying cutoff times. The table below summarizes the performance:

Dataset	Time(s)	Solution Quality	Full Tour	Rel Err
Atlanta.tsp	900	2034195.0	No	1.52%
Berlin.tsp	900	9124.0	No	9.03%
Boston.tsp	900	1006492.0	No	9.40%
Champaign.tsp	900	55948.0	No	2.33%
Cincinnati.tsp	900	277952.0	Yes	0.00%
Denver.tsp	900	115551.0	No	9.50%
NYC.tsp	900	1784453.0	No	6.86%
Philadelphia.tsp	900	1419453.0	No	1.68%
Roanoke.tsp	900	834230.0	No	-20.34%
SanFrancisco.tsp	900	1045182.0	No	8.15%
Toronto.tsp	900	1369024.0	No	-21.58%
UKansasState.tsp	900	62962.0	Yes	0.00%
UMissouri.tsp	900	164593.0	No	-1.07%

Table 1: Brute-Force Algorithm Performance

The solution quality of the Brute-Force with Branch-and-Bound algorithm is directly influenced by the cutoff time. A longer cutoff time allows the algorithm to explore more routes, resulting in better solutions as the algorithm has more opportunities to find and refine the optimal path. Conversely, shorter cutoff times may result in premature termination, yielding suboptimal solutions.

Additionally, the inclusion of Branch-and-Bound significantly improves solution quality compared to an original brute-force approach without it. As shown in the plots, the "with Branch-and-Bound" algorithm consistently produces lower-cost solutions across all cutoff times, leveraging effective pruning to eliminate suboptimal routes early.

As shown in Figure 1, the relationship is demonstrated, showing the improvement in solution quality as the cutoff time increases:

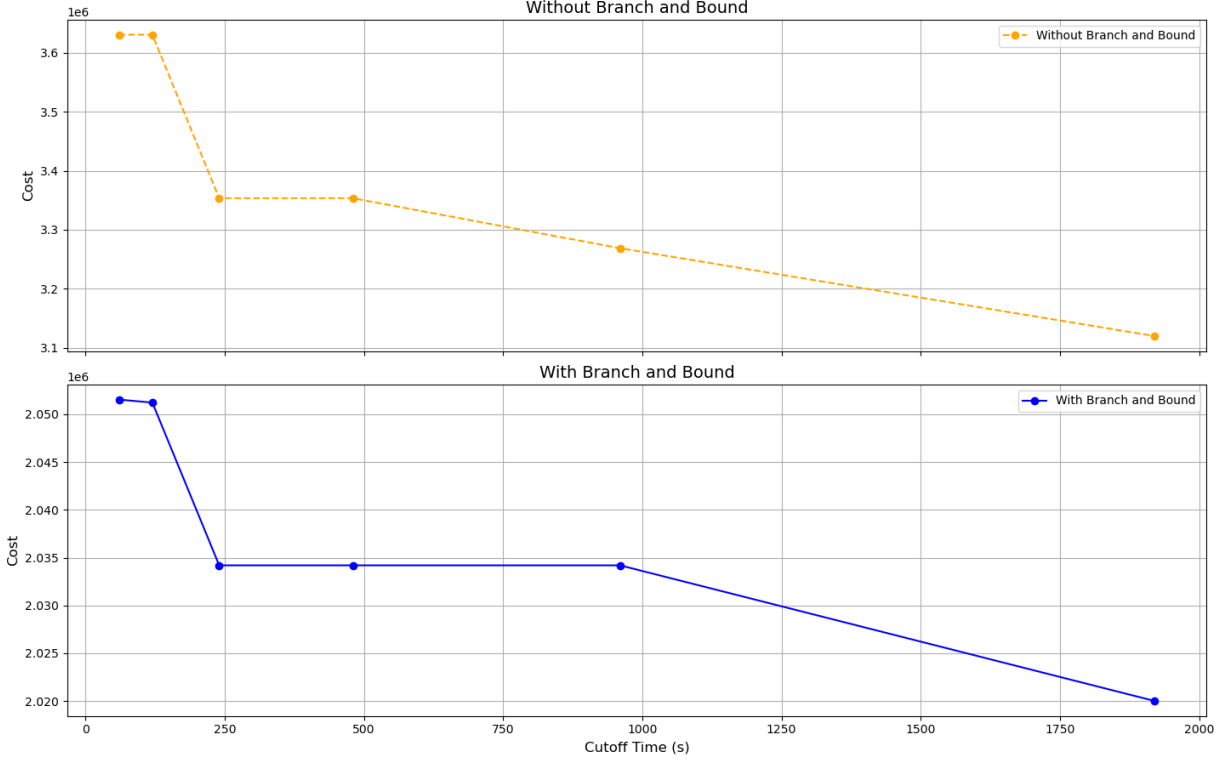


Figure 1: Effect of Cutoff Time on Solution Quality: With vs. Without Branch-and-Bound.

2.2 2-Approximation Based on Minimum Spanning Tree (MST)

The route generated by performing a depth-first search (DFS) on the Minimum Spanning Tree (MST) provides a **2-approximation** to the solution of the Metric TSP. This approximation guarantee is a direct consequence of the triangle inequality. A brief proof is outlined below.

Let the optimal solution to the Metric TSP be the route R , with a total cost denoted as $C(R)$. If we remove any edge from R , the remaining edges form a spanning tree whose total cost is greater than or equal to the cost of the MST, denoted as $C(MST)$. Thus, $C(MST) \leq C(R)$.

Now, consider traversing the MST in a depth-first order, starting from an arbitrary root node:

- If we follow the tree edges exactly, visiting and returning to nodes as in a standard DFS traversal, the total cost is twice the total cost of the MST, $2C(MST)$, since each edge is traversed once in each direction.
- If we "shortcut" the traversal, skipping revisits to previously visited nodes (as permitted by the triangle inequality), the resulting route (a DFS traversal of the MST) will have a total cost less than $2C(MST)$.

Therefore, the cost of the DFS route is less than $2C(MST)$, and since $C(MST) \leq C(R)$, the cost of the DFS route is less than $2C(R)$. This establishes that the DFS route of the MST provides a solution that is at most twice the cost of the optimal Metric TSP solution.

The DFS traversal of the MST can start from any node chosen as the root, resulting in at most n possible routes (n is the number of nodes). Each of these routes is a 2-approximation of the optimal solution. Among them, we select the route with the smallest total cost as the final result.

The 2-approximation algorithm was tested on different datasets. The table below summarizes the performance:

Dataset	Time(s)	Solution Quality	Full Tour	Rel Err
Atlanta.tsp	1.9517×10^{-4}	2314410.0	Yes	15.50%
Berlin.tsp	1.0040×10^{-3}	9794.0	Yes	17.04%
Boston.tsp	6.0333×10^{-4}	1089714.0	Yes	18.45%
Champaign.tsp	1.0901×10^{-3}	64529.0	Yes	18.02%
Cincinnati.tsp	5.5833×10^{-5}	290257.0.0	Yes	4.43%
Denver.tsp	2.8796×10^{-3}	123680.0	Yes	17.20%
NYC.tsp	2.2778×10^{-3}	1972735.0	Yes	18.13%
Philadelphia.tsp	3.9817×10^{-4}	1515175.0	Yes	8.54%
Roanoke.tsp	2.1825×10^{-2}	789717.0	Yes	-24.59%
SanFrancisco.tsp	3.8399×10^{-3}	1092009.0	Yes	12.99%
Toronto.tsp	4.5555×10^{-3}	1520072.0	Yes	-12.92%
UKansasState.tsp	4.8750×10^{-5}	66529.0	Yes	5.67%
UMissouri.tsp	5.3650×10^{-3}	157926.0	Yes	-5.08%

Table 2: 2-Approximation Algorithm Performance

2.3 Local Search - Simulated Annealing

For the local search section, we've implemented the Simulated Annealing Algorithm, which, as an extension of Metropolis Algorithm, is also inspired by principles from thermal physics. The idea of Simulated Annealing algorithm is similar to the general structure of other local research methods-

1. Start from a randomly chosen solution S
2. Apply a small perturbation to the current solution S to find a neighboring solution S'
3. If S' has lower cost than S , then we update S' to be new current solution
4. Repeat 2 & 3 above until we've reached targeted precision or the early stopping criteria

Specifically, for the Simulated Annealing Algorithm, the pseudo-code is as follows, where the `dist(route)` is used to calculate the total distance (i.e., solution quality) of a given route, and `swap(route)` is used to randomly swap a pair of nodes in the route (i.e., to generate neighboring solutions).

Algorithm 1 `sim_ann(k, M, ini_temp, cool_rate, max_iter, cutoff, seed)`

```
if seed  $\neq$  None then
    Set all the random seeds to seed
end if
res_route = Randomly initialize a route
res_dist = dist(res_route)
temp = ini_temp ▷ Set Initial Temperature
for iteration in range(max_iter) do
    elapsed = Calculate current time used
    if elapsed > cutoff then
        break
    end if
    temp_route = swap(res_route) ▷ Generate Neighboring Solution
    temp_dist = dist(temp_route)
    if temp_dist < res_dist then
        res_route = temp_route
        res_dist = temp_dist ▷ Update Solution
    else
         $\Delta E = \text{temp\_dist} - \text{res\_dist}$ 
         $p = e^{-\frac{\Delta E}{k \cdot \text{temp}}}$  ▷ Probability of Accepting Higher Cost Solution
        rand_num = Generate random number between 0 and 1
        if rand_num < p then
            res_route = temp_route
            res_dist = temp_dist ▷ Accept Higher Cost Solution
        end if
    end if
    if iteration % M == 0 then
        temp = temp  $\times$  cool_rate ▷ Lower the Temperature Every M Steps
    end if
end for
return res_route, res_dist
```

In our experiments, as expected, the result of the Simulated Annealing Algorithm can depend on the detailed "cooling schedule" (i.e., the hyperparameters) being set. Here we have chosen $k = 1.5$, $M = 1000$, $\text{ini_temp} = 10000$, $\text{cool_rate} = 0.99$, $\text{max_iter} = 1000000$ (since we want to use cutoff time to control), $\text{cutoff} = 900$, $\text{seed} = 42$, and the corresponding results are summarized as follows-

Dataset	Time(s)	Solution Quality	Full Tour
Atlanta.tsp	900	2003763.0	No
Berlin.tsp	900	8368.0	No
Boston.tsp	900	920007.0	No
Champaign.tsp	900	54676.0	No
Cincinnati.tsp	900	277952.0	Yes
Denver.tsp	900	105530.0	No
NYC.tsp	900	1669947.0	No
Philadelphia.tsp	900	1395981.0	No
Roanoke.tsp	900	1047195.0	No
SanFrancisco.tsp	900	966434.0	No
Toronto.tsp	900	1745650.0	No
UKansasState.tsp	900	62962.0	Yes
UMissouri.tsp	900	166371.0	No

Table 3: Local Search Algorithm Performance